grow). Instead, I'm talking about writing user-space programs—including the exercises, homework, and project work that most computer-science study courses include.

Before we start, here's a disclaimer: this article contains strong personal opinions and beliefs; I do not in any way intend to be offensive, but some of these ideas just might be worth a try—by you—to see if you feel the same way!

Attacking the mindset

It's commonly believed that Linux is 'tough'. Sure, it's different from what people who're used to Windows are accustomed to—but it's not tough. Once you adjust to the differences, you'll probably laugh at this misconception yourself, and tell others how wrong their perception is!

Just consider the many computer science students who've been inspired by the buzz that Linux has been creating over a long time now. They have resolutely set about learning how to use it *on their own initiative*—asking questions on mailing lists, forums and over IRC chat. Within a couple of weeks, they are ready to do more than just get around. Often, within a month, they're so much at home with Linux that they begin introducing others to the OS. Astounding? It may seem so—but it's just that those students were determined to explore and learn, and ignored the cries of, "It's tough."

There is always a learning curve involved whenever one is acquiring a new skill, and Linux is no exception. If students are taught to use and program on Linux, they will not just learn, but will also find it simple. It would just seem natural to them—learning something that they did not know earlier. 'Linux is tough' is a modern-day myth that has to be busted. If you are an educator, please do your bit. You are the one that students look up to, and if you show them the right way, they will follow your example.

Getting Linux up and running

Okay, once you have decided to use Linux, how do you go about it? You may have heard of lots of different Linux 'operating systems' (also called distributions): Ubuntu Linux, Fedora Linux, Debian GNU/Linux and more. Why so many 'Linuxes'? Let me explain. Technically, 'Linux' is the name of a kernel (read http://en.wikipedia.org/wiki/Linux_kernel for more information; see http://www.kernel.org, which is the official home of the kernel). Since a kernel is of little use on its own, user-space tools from the GNU project (including the most common implementation of the C library, a popular shell, and many common UNIX tools that carry out many basic operating system tasks) were combined with the Linux kernel to make a usable operating system. The graphical user interface (or GUI) used by most Linux systems is built on top of an implementation of the X Window System. Different free software projects and vendors build different combinations of packages and features, to provide varying Linux experiences to different target audiences—thus resulting in myriad Linux distributions.

So which Linux distribution should you use? Ubuntu Linux (http://www.ubuntu.com) and Fedora Linux (http://www.fedoraproject.org) both have individually made the Linux experience very user-friendly for casual users of the computer—for Internet surfing, e-mail and document processing needs. Either of these is ideal for you to get started with. Linux installation can be somewhat tricky, though, especially if you intend to set up a *dual-boot system* where you can boot either Linux or your old Windows. Otherwise, it's quite simple: download the CD (ISO) image, burn it to a CD-R or RW, boot your computer from it, and let it install! The best way to do a dual-boot set-up the first time is to get hold of someone in your school, locality or office who knows about it, and ask them to guide you. Also, there are other options if you want to try Linux either without installing it, without replacing Windows or doing a dual-boot set-up. See the *Dealing with practicalities* section towards the end of this article, for some of these ideas.

The *Ultimate Linux Newbie Guide* (http://www.linuxnewbieguide.org/) is a good reference to help you learn things yourself. With Linux, an experimental approach to learning helps a lot. So, back up your data, and get started with those install discs if you can't find anyone to help you out.

These days, most Linux distributions come with just the essential applications and libraries installed—which probably won't be sufficient for programming needs. To enable easy installation of new software, most distributions have a *package manager* (in the Linux world, software is distributed in the form of 'packages'), which you use to easily download and install new software from the Internet. The Linux Newbie Guide mentioned earlier is a good reference for this topic.

So that this article will be of maximum utility, I will try to be more general, and avoid favouring any particular distribution.

Choosing a text editor

We won't be using an Integrated Development Environment (IDE), at least, initially. We will just do it the simple way: write code using a text editor, save it, and compile/interpret it using an appropriate compiler/interpreter. In the Linux world, you have a plethora of text editors to choose from. One of the editors, such as *gedit* or *kwrite*, will definitely be installed when you install Linux—you can use either. If you install a distribution like Ubuntu, which has the GNOME desktop environment, then you will have *gedit* already installed. It's just like Notepad, only more useful and feature-rich.

C/C++ programming on Linux

C is usually the first language taught to many students in Indian engineering schools and colleges, so let's first look at how we program in C on Linux. Note that the C code that you will write on Linux will be the same that you would write on Windows/DOS, as long as you are writing ANSI C code. Some library functions, such as those provided by *conio.h* and *graphics.h*, are not part of the ANSI standard. Hence, you won't be able to use them on Linux. The C compiler you use on Linux is GCC. It is part of the GNU Compiler Collection (http://en.wikipedia.org/wiki/GNU_Compiler_Collection).